PROGRAMACIÓN V · UNIDAD III · PATRONES DE DISEÑO

Patrones de diseño

Ocho patrones con ejemplos clásicos en Java que se compilan, se corren y se testean, listos para llevar a su proyecto

1 Factory

2 State

3 Chain

4 Strategy

5 Repository

6 Adapter

7 Observer

8 Facade

Prof. Lic. Aguirre, Juan José · Licenciatura en Sistemas · FCAD-UNER · 2026

Un patrón es un nombre para una decisión

La pregunta de la defensa no es
«¿qué patrones usaron?»

sino
«¿por qué este patrón acá?»
«¿qué pasa si dos lo hacen a la vez?»
«¿qué hay que tocar para agregar uno nuevo?»

Hoy cada patrón responde alguna de esas preguntas con un ejemplo de libro que pueden correr.

1

¿Qué problema resuelve?

Cada patrón arranca de un problema concreto, no del catálogo.

2

¿Cómo se ve en código?

Java 21 ejecutable: misma salida en el proyector que en su máquina.

3

¿Cuándo NO usarlo?

El costo se dice siempre. Justificar un descarte también puntúa.

7’ encuadre · 48’ ocho patrones (6’ c/u) · 5’ cierre

MAPA DE LA CLASE

Ocho problemas que van a tener en su proyecto

1 Factory Method
CREACIONAL

Ejemplo: notificadores por canal.

En su proyecto: crear objetos según un tipo que llega como texto.

2 State
COMPORTAMIENTO

Ejemplo: documento de borrador a publicado.

En su proyecto: pedidos, turnos, tickets: entidades con ciclo de vida.

3 Chain of Resp.
COMPORTAMIENTO

Ejemplo: aprobación de compras por monto.

En su proyecto: aprobaciones por nivel y filtros encadenados.

4 Strategy
COMPORTAMIENTO

Ejemplo: costo de envío intercambiable.

En su proyecto: precios, tarifas y descuentos que cambian seguido.

5 Repository
APLICACIÓN

Ejemplo: biblioteca con préstamos simultáneos.

En su proyecto: acceso a datos y dos usuarios sobre el mismo registro.

6 Adapter
ESTRUCTURAL

Ejemplo: sensores de terceros en °F y en XML.

En su proyecto: pasarelas de pago y APIs externas.

7 Observer
COMPORTAMIENTO

Ejemplo: estación meteorológica.

En su proyecto: notificaciones, eventos de dominio y auditoría.

8 Facade
ESTRUCTURAL

Ejemplo: compra en una tienda online.

En su proyecto: el servicio que orquesta un caso de uso completo.

Un proyecto, ocho paquetes, una demo por patrón

ESTRUCTURA

```
patrones-java/
├── run.sh · run.bat · pom.xml · README.md
└── src/main/java/ar/edu/uner/progv/patrones/
    ├── Main.java          menú de demos
    ├── comun/             Formato (montos es-AR)
    ├── factory/           1 · notificadores
    ├── state/             2 · documento
    ├── chain/             3 · aprobación de compras
    ├── strategy/          4 · costo de envío
    ├── repository/        5 · biblioteca
    ├── adapter/           6 · sensores
    │   └── externo/           "código ajeno"
    ├── observer/          7 · estación meteorológica
    └── facade/            8 · tienda online
src/test/java/...          un test por patrón
```

`./run.sh`

Compila con javac y corre las ocho demos en el orden de hoy. En Windows: run.bat

`./run.sh strategy`

Una sola demo: factory · state · chain · strategy · repository · adapter · observer · facade

`mvn test`

39 tests JUnit 5, uno por regla. El de concurrencia corre 10 veces

Java 21 puro: sin Spring ni base de datos, para ver el patrón sin ruido. Montos en formato es-AR fijo: la salida es idéntica en todas las máquinas.

1

Factory Method

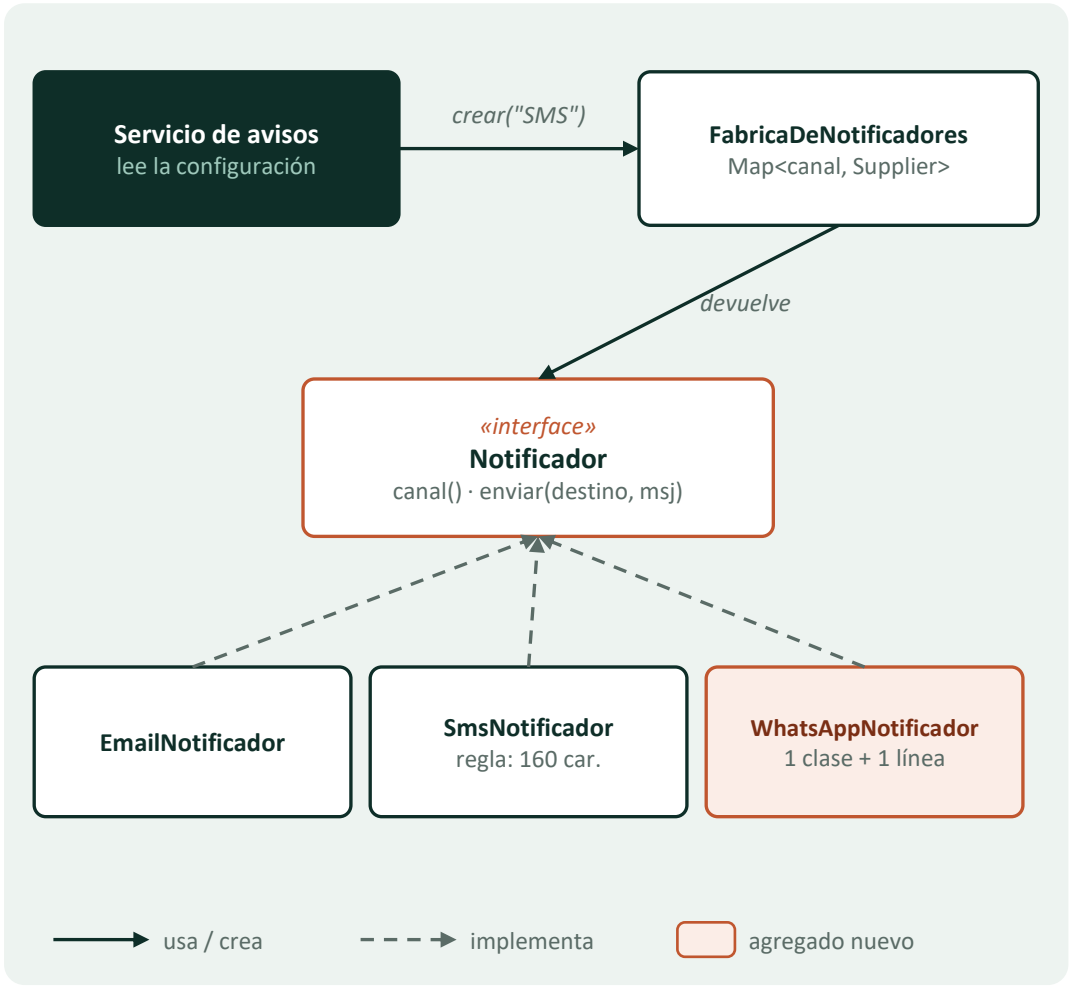
Intención *Que el sistema cree objetos sin conocer sus clases concretas.*

El problema

Un sistema avisa por email, SMS o push, y el canal sale de la configuración o de la preferencia del usuario. Mañana piden WhatsApp. ¿Cuántos archivos hay que tocar?

ANTES · sin el patrón

```
void avisar(String canal, String d, String m) {
    switch (canal) {
        case "EMAIL" -> new EmailNotificador().enviar(d, m);
        case "SMS"   -> new SmsNotificador().enviar(d, m);
        case "PUSH"  -> new PushNotificador().enviar(d, m);
        // ¿WhatsApp? tocar este switch y cada copia suya
        default -> throw new IllegalArgumentException(canal);
    }
}
```



1

Factory Method en código

DESPUÉS

```
public interface Notificador {
    String canal();
    String enviar(String destino, String mensaje);
}

public final class FabricaDeNotificadores {
    private final Map<String, Supplier<Notificador>> creadores = new LinkedHashMap<>();

    public static FabricaDeNotificadores conCanalesEstandar() {
        return new FabricaDeNotificadores()
                .registrar("EMAIL", EmailNotificador::new)      // el constructor
                .registrar("SMS", SmsNotificador::new)          // es el creador
                .registrar("PUSH", PushNotificador::new);
    }

    public FabricaDeNotificadores registrar(String canal, Supplier<Notificador> c) {
        creadores.put(canal.trim().toUpperCase(), c);
        return this;
    }

    public Notificador crear(String canal) {
        Supplier<Notificador> creador = creadores.get(canal.trim().toUpperCase());
        if (creador == null)
            throw new IllegalArgumentException("Canal desconocido: " + canal);
        return creador.get();
    }
}
```

```
$ ./run.sh factory
[OK] email a ana: "Tu pedido salió"
[OK] sms a ana (15 car.)
[X]  Canal desconocido: WhatsApp
... registrar("WHATSAPP",
      WhatsAppNotificador::new)
[OK] email · sms · whatsapp
```

Cuándo NO usarlo

- Un solo canal y ninguno más a la vista: un new alcanza.
- Si los tipos solo cambian datos y no comportamiento, es una columna, no una subclase. SMS sí la merece: tiene su regla de 160 caracteres.

2

State

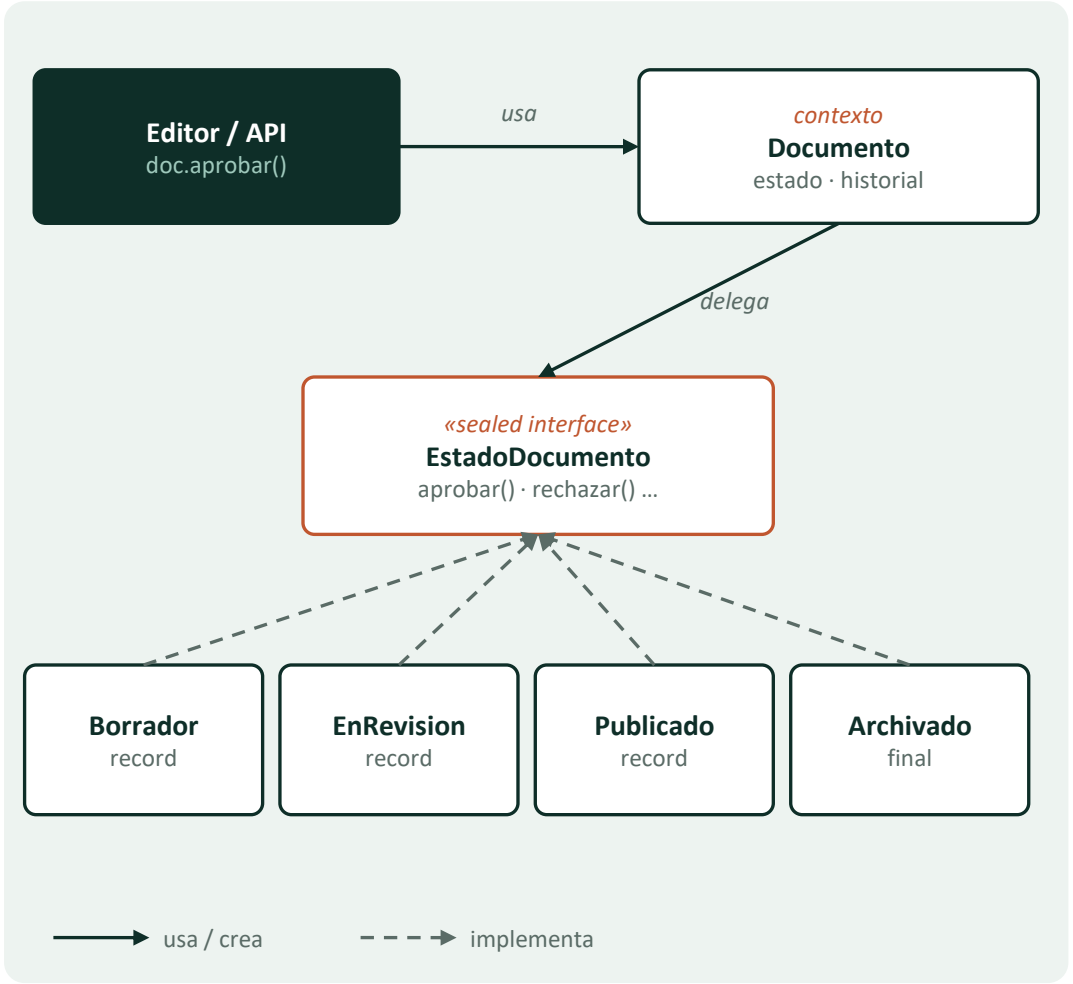
Intención *Que el comportamiento cambie cuando cambia el estado interno, sin cadenas de if.*

El problema

Un documento pasa por Borrador, En revisión, Publicado y Archivado. Según el estado se puede editar o no, y cada acción lleva a un estado distinto… o no está permitida.

Transiciones: quién decide es el estado, no el que llama

Acción	BORRADOR	EN_REVISION	PUBLICADO	ARCHIVADO
editar	sí	✗	✗	✗
enviar a rev.	→ REVISIÓN	✗	✗	✗
aprobar	✗	→ PUBLICADO	✗	✗
rechazar	✗	→ BORRADOR	✗	✗
archivar	→ ARCHIVADO	✗	→ ARCHIVADO	✗



2

State en código

DESPUÉS

```
public sealed interface EstadoDocumento
        permits Borrador, EnRevision, Publicado, Archivado {
    String nombre();
    default boolean permiteEditar()            { return false; }
    default EstadoDocumento enviarARevision()  { throw invalida("enviar a revisión"); }
    default EstadoDocumento aprobar()          { throw invalida("aprobar"); }
    default EstadoDocumento rechazar()         { throw invalida("rechazar"); }
    default EstadoDocumento archivar()         { throw invalida("archivar"); }
}

public record EnRevision() implements EstadoDocumento {    // solo lo que acepta
    public String nombre()             { return "EN_REVISION"; }
    public EstadoDocumento aprobar()   { return new Publicado(); }
    public EstadoDocumento rechazar()  { return new Borrador(); }
}

public final class Documento {                     // contexto: ni un if por estado
    private EstadoDocumento estado = new Borrador();

    public void aprobar() { cambiar(estado.aprobar()); }        // lanza si no corresponde

    public void editar(String texto) {
        if (!estado.permiteEditar())
            throw new TransicionInvalidaException(estado.nombre(), "editar");
        contenido = texto;
    }
}
```

```
$ ./run.sh state

[OK] editar v2       -> BORRADOR
[OK] enviar a rev.   -> EN_REVISION
[X]  editar          -> no se puede
[OK] rechazar        -> BORRADOR
[OK] editar · enviar · aprobar
                     -> PUBLICADO
[X]  rechazar        -> no se puede
[OK] archivar        -> ARCHIVADO
```

Cuándo NO usarlo

- Dos estados y una regla: un boolean se lee mejor.
- Si el negocio define los flujos en runtime, las transiciones van en una tabla, no en clases.

3

Chain of Responsibility

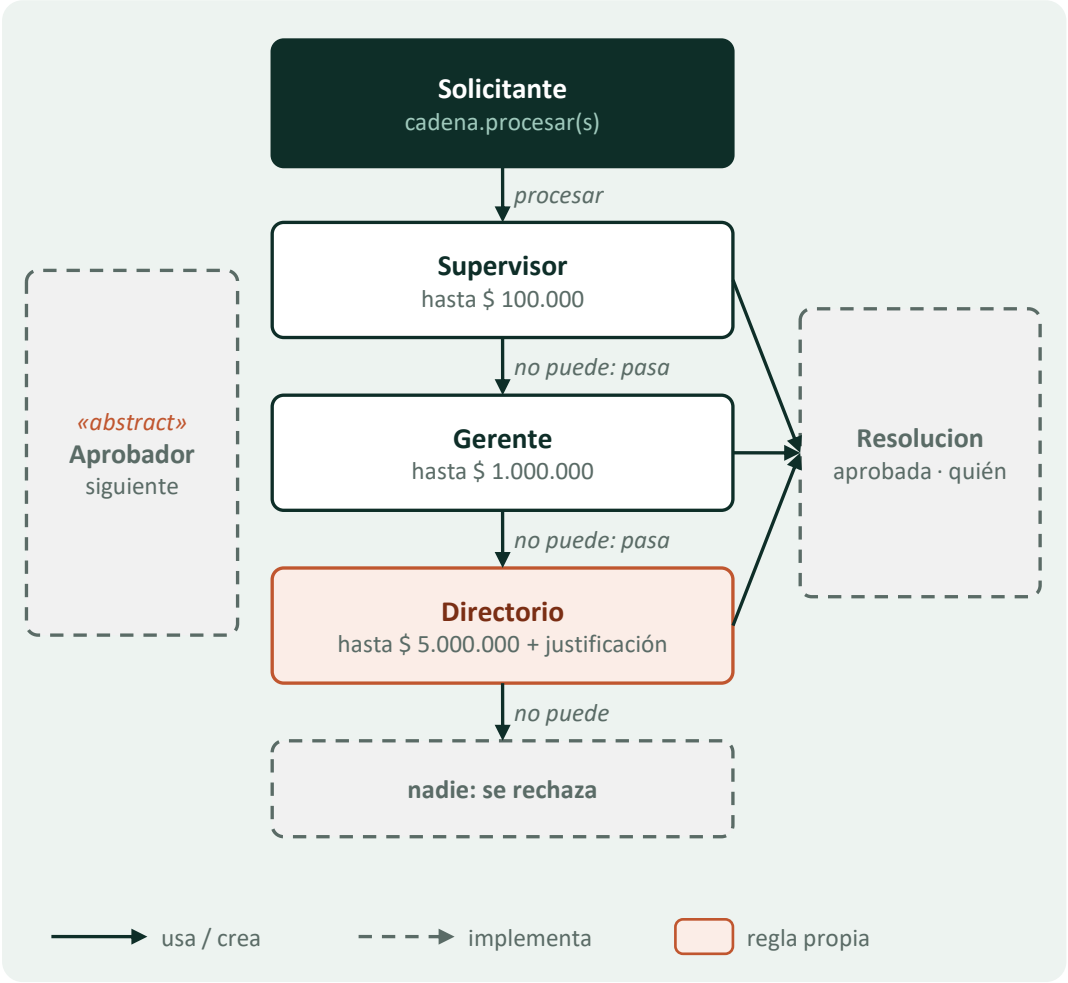
Intención *Pasar una solicitud por una cadena de manejadores; el primero que puede, la resuelve.*

El problema

Una compra la aprueba el Supervisor hasta $ 100.000, el Gerente hasta $ 1.000.000 y el Directorio hasta $ 5.000.000, y solo si viene justificada. Los niveles y los límites cambian.

ANTES · sin el patrón

```
Resolucion aprobar(SolicitudDeCompra s) {
    if (s.monto() <= 100_000)   return ok("Supervisor");
    if (s.monto() <= 1_000_000) return ok("Gerente");
    if (s.monto() <= 5_000_000)
        return s.justificacion().isBlank()
                ? rechazo("Directorio") : ok("Directorio");
    return rechazo("nadie");
    // ¿un nivel nuevo, o cambia un límite? tocar acá
}
```



3

Chain of Responsibility en código

DESPUÉS

```
public abstract class Aprobador {
    private Aprobador siguiente;

    public Aprobador enlazar(Aprobador siguiente) {
        this.siguiente = siguiente;
        return siguiente;
    }

    public final Resolucion procesar(SolicitudDeCompra s) {
        if (puedeResolver(s)) return resolver(s);              // la resuelve y termina
        if (siguiente != null) return siguiente.procesar(s);   // o la pasa
        return new Resolucion(false, "nadie", "ningún nivel puede aprobar ese monto");
    }

    protected abstract boolean puedeResolver(SolicitudDeCompra s);
    protected abstract Resolucion resolver(SolicitudDeCompra s);
}

public class AprobadorPorMonto extends Aprobador {
    protected boolean puedeResolver(SolicitudDeCompra s) {
        return s.monto().compareTo(limite) <= 0;
    }
}

Aprobador cadena = Aprobador.cadena(
        new AprobadorPorMonto("Supervisor", new BigDecimal("100000")),
        new AprobadorPorMonto("Gerente", new BigDecimal("1000000")),
        new Directorio(new BigDecimal("5000000")));          // exige justificación
```

```
$ ./run.sh chain

[OK] C-1 $ 45.000      -> Supervisor
[OK] C-2 $ 380.000     -> Gerente
[OK] C-3 $ 3.200.000   -> Directorio
[X]  C-4 sin justificación
                       -> Directorio
[X]  C-5 $ 90.000.000 -> nadie
```

Cuándo NO usarlo

- Si los niveles solo cambian un número, una tabla de límites ordenada alcanza: fíjense que Supervisor y Gerente son la misma clase.
- Si la solicitud debe pasar por todos (validaciones), es un pipeline: cada eslabón verifica y sigue.

4

Strategy

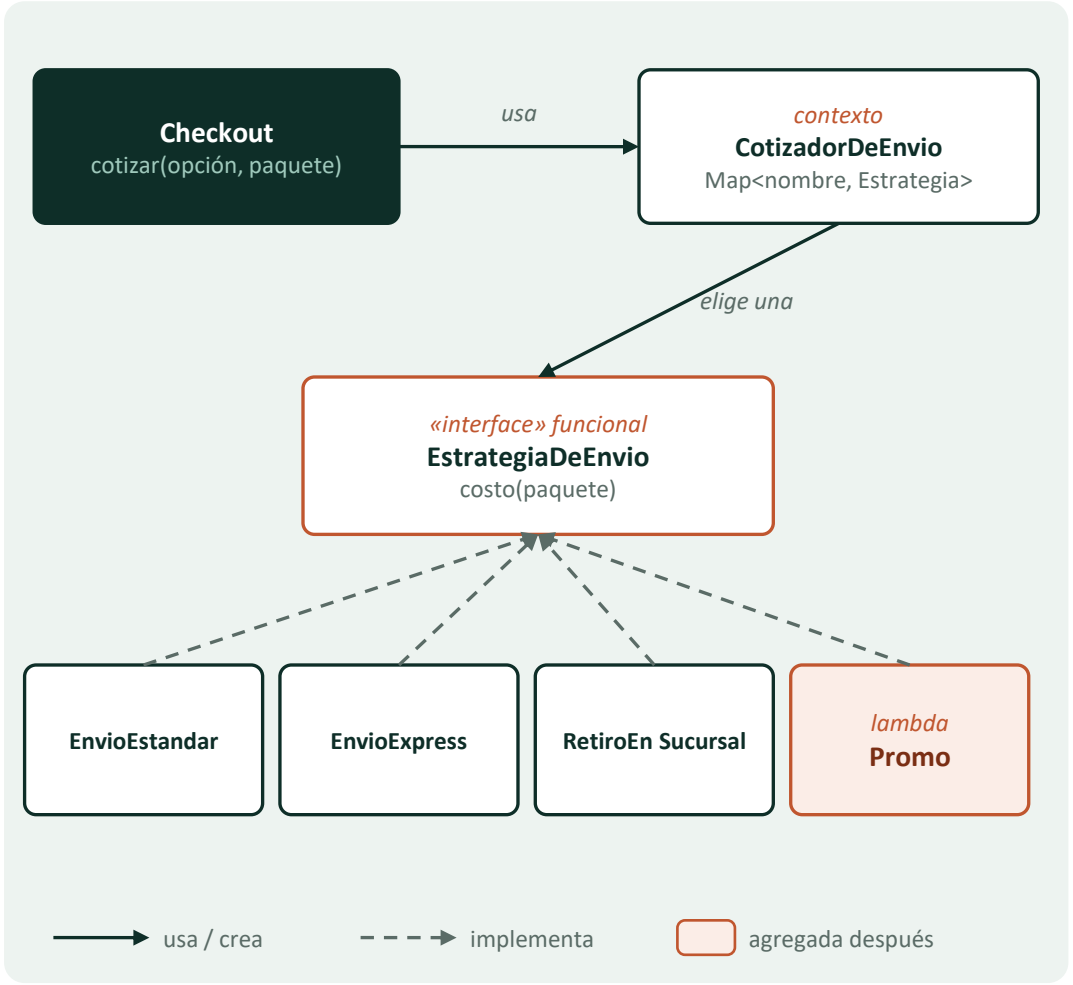
Intención *Encapsular cada variante de un algoritmo en su propia clase y hacerlas intercambiables.*

El problema

Una tienda ofrece envío estándar, express y retiro en sucursal, y cada uno se calcula distinto. Marketing ya avisó que van a venir promociones, envío en moto y envío internacional.

ANTES · sin el patrón

```
BigDecimal costoEnvio(String opcion, Paquete p) {
    switch (opcion) {
        case "ESTANDAR": return base(p);
        case "EXPRESS":  return base(p).multiply(RECARGO)
                                       .add(seguro(p));
        case "RETIRO":   return BigDecimal.ZERO;
        // promo, moto en el día, internacional...
        default: throw new IllegalArgumentException(opcion);
    }
}
```



4

Strategy en código

DESPUÉS · extracto

```java
@FunctionalInterface
public interface EstrategiaDeEnvio {
    BigDecimal costo(Paquete paquete);
}

public final class EnvioEstandar implements EstrategiaDeEnvio {
    public BigDecimal costo(Paquete p) {                    // base + por kilo + por km
        return BASE.add(POR_KG.multiply(p.pesoKg()))
                   .add(POR_KM.multiply(BigDecimal.valueOf(p.distanciaKm())));
    }
}

public final class CotizadorDeEnvio {                      // el contexto
    private final Map<String, EstrategiaDeEnvio> opciones = new LinkedHashMap<>();

    public CotizadorDeEnvio agregar(String nombre, EstrategiaDeEnvio estrategia) {
        opciones.put(nombre, estrategia);
        return this;
    }

    public BigDecimal cotizar(String opcion, Paquete paquete) {
        return opciones.get(opcion).costo(paquete).setScale(2, RoundingMode.HALF_UP);
    }
}

// opción nueva, cero archivos modificados: una lambda
cotizador.agregar("Promo", p -> p.valorDeclarado().compareTo(CIEN_MIL) > 0
        ? BigDecimal.ZERO : estandar.costo(p));
```

```
$ ./run.sh strategy

libro 0,5 kg a 40 km
  Estándar $ 3.800  · Express $ 7.040
  Retiro $ 0
TV 12 kg a 300 km
  Estándar $ 12.300  · Express $ 28.140
  Retiro $ 0
+ lambda «Promo»
  libro $ 3.800  · TV $ 0
```

Cuándo NO usarlo

- Una sola forma de calcular que nunca cambia: un método alcanza.
- Si las reglas se combinan (descuento + recargo) en vez de elegirse una, es una lista de políticas aplicadas en orden.

5

Repository

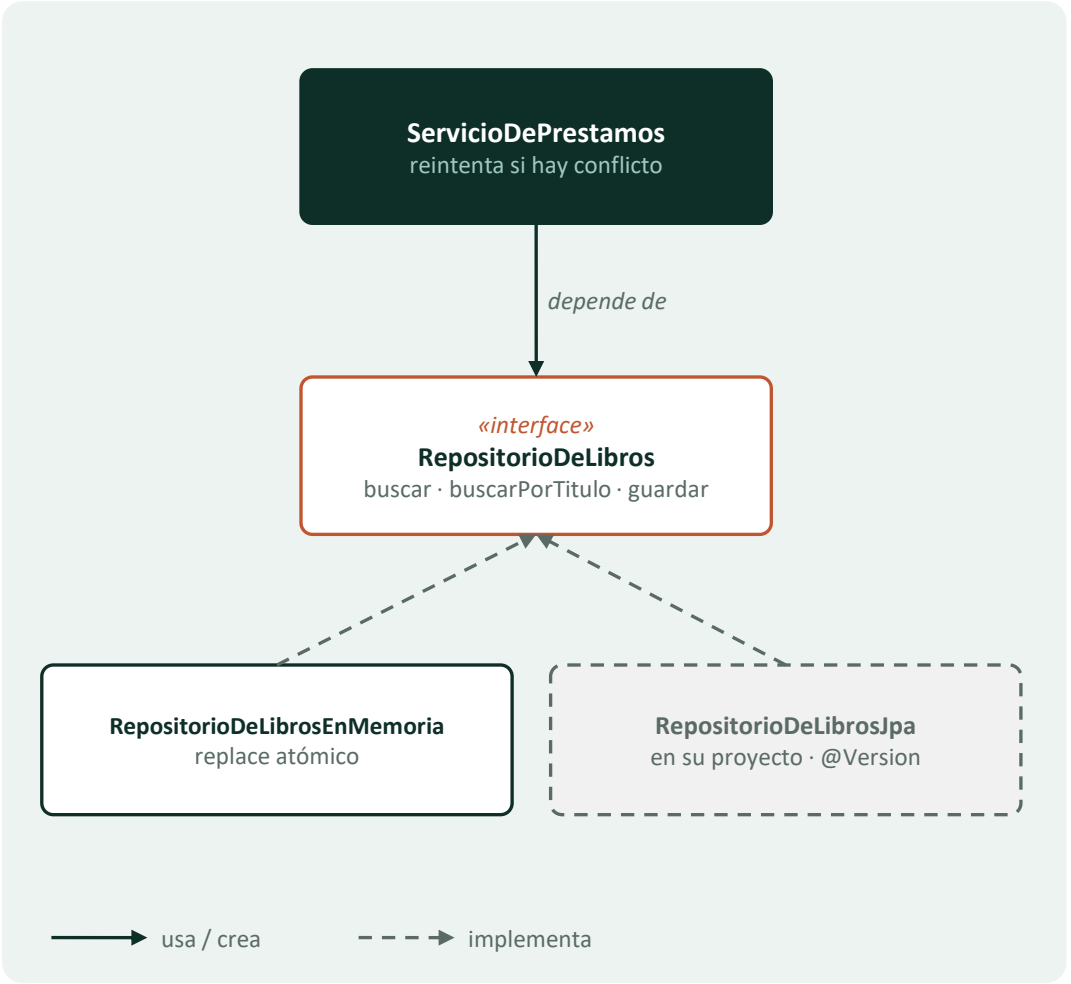
Intención *Que el negocio vea una colección de objetos y no sepa cómo ni dónde se guardan.*

El problema

Una biblioteca presta libros. Si quedan 3 ejemplares y 20 lectores piden el mismo libro a la vez, tienen que salir exactamente 3 préstamos. Y el servicio no debería saber si hay una base, memoria o un mock.

ANTES · sin el patrón

```
public void prestar(String isbn) {
    Integer disp = jdbc.queryForObject(
        "SELECT disponibles FROM libro WHERE isbn=?", ...);
    if (disp == 0) throw new SinEjemplaresException(...);
    jdbc.update("UPDATE libro SET disponibles=? WHERE isbn=?",
                disp - 1, isbn);
    // SQL en el negocio + carrera: dos lectores leen 1,
    // los dos escriben 0 y se prestan 2 ejemplares de 1
}
```



5

Repository en código

DESPUÉS · extracto

```
public interface RepositorioDeLibros {              // el negocio ve una colección
    Optional<Libro> buscar(String isbn);
    List<Libro> buscarPorTitulo(String texto);
    Libro guardar(Libro libro);                      // ConflictoDeVersion si quedó viejo
}

// en memoria: replace(clave, esperado, nuevo) hace de UPDATE ... WHERE version = ?
public Libro guardar(Libro libro) {
    Libro actual = libros.get(libro.isbn());
    if (actual.version() != libro.version())
        throw conflicto(libro, actual);
    Libro nuevo = libro.conVersion(actual.version() + 1);
    if (!libros.replace(libro.isbn(), actual, nuevo))          // atómico
        throw conflicto(libro, actual);
    return nuevo;
}

// el servicio solo conoce la interfaz y reintenta ante un conflicto
public Libro prestar(String isbn) {
    for (int intento = 1; intento <= MAX_INTENTOS; intento++) {
        Libro leido = repositorio.buscar(isbn).orElseThrow();
        try { return repositorio.guardar(leido.prestarUno()); }  // SinEjemplares corta
        catch (ConflictoDeVersionException e) { /* releer y reintentar */ }
    }
    throw new IllegalStateException("Demasiada contención");
}
```

```
$ ./run.sh repository

buscarPorTitulo("design")
  -> Design Patterns
20 pedidos simultáneos
  -> 3 préstamos, 17 sin ejemplares
quedan 0 disponibles · versión 4
[X] guardar una copia vieja
  -> leíste v1 y ya va por la v2
```

Cuándo NO usarlo

- No envuelvas Spring Data por envolver: si JpaRepository ya expresa lo que el negocio necesita, usalo directo.
- El patrón no evita la carrera: la evita el almacenamiento (@Version, UNIQUE o un lock).

6

Adapter

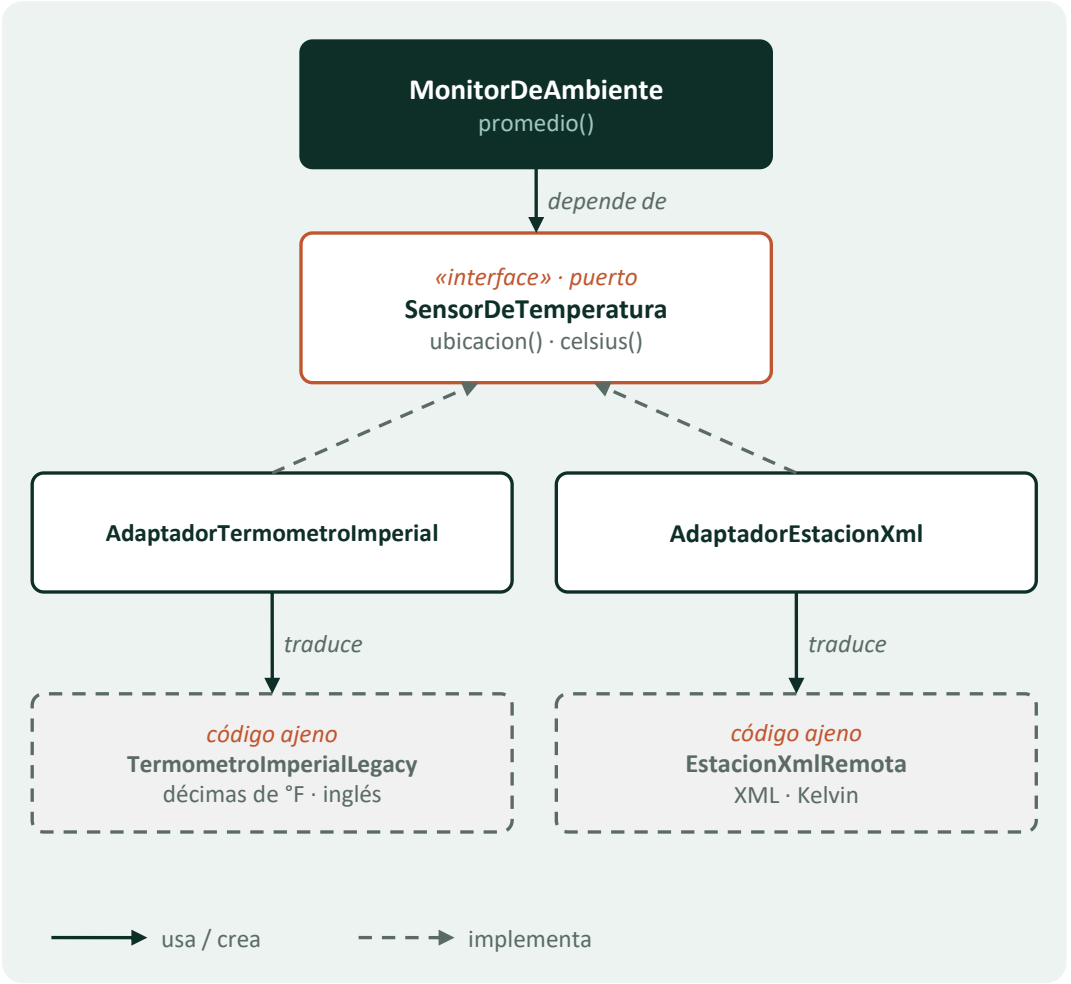
Intención *Convertir la interfaz de una clase ajena en la interfaz que el sistema espera.*

El problema

Un monitor de ambiente promedia sensores de distintos fabricantes: uno da décimas de °F con una API en inglés, otro responde XML en Kelvin. Ninguno de los dos se puede modificar.

ANTES · sin el patrón

```
double promedio() {
    double f = termometro.readTenthsFahrenheit() / 10.0;
    double c1 = (f - 32) * 5 / 9;
    String xml = estacion.consultar();
    double k = Double.parseDouble(xml.substring(
            xml.indexOf('>') + 1, xml.indexOf("</")));
    double c2 = k - 273.15;
    return (oficina.celsius() + c1 + c2) / 3;
    // conversiones y parseo mezclados con el negocio
}
```



6

Adapter en código

DESPUÉS

```java
public interface SensorDeTemperatura {              // puerto: nuestro idioma, en °C
    String ubicacion();
    double celsius();
}

public final class AdaptadorTermometroImperial implements SensorDeTemperatura {
    private final TermometroImperialLegacy termometro;     // código ajeno, no se toca

    public double celsius() {
        double fahrenheit = termometro.readTenthsFahrenheit() / 10.0;
        return (fahrenheit - 32) * 5 / 9;                  // la conversión vive solo acá
    }
}

public final class AdaptadorEstacionXml implements SensorDeTemperatura {
    public double celsius() {
        return Double.parseDouble(leer().group(2)) - 273.15;   // XML y Kelvin -> °C
    }
}

public final class MonitorDeAmbiente {              // el cliente solo conoce el puerto
    public double promedio() {
        return sensores.stream().mapToDouble(SensorDeTemperatura::celsius)
                .average().orElse(Double.NaN);
    }
}
```

```
$ ./run.sh adapter

Oficina           22,5 °C (nativo)
Depósito [TX-90]  24,0 °C (de °F)
Terraza           30,5 °C (de XML/K)
promedio          25,7 °C
```

Cuándo NO usarlo

- Si la API ajena ya habla tu idioma, el adaptador es una capa vacía: el sensor de la oficina no lo necesita.
- Nunca pongas reglas de negocio en el adaptador: solo traduce.

7

Observer

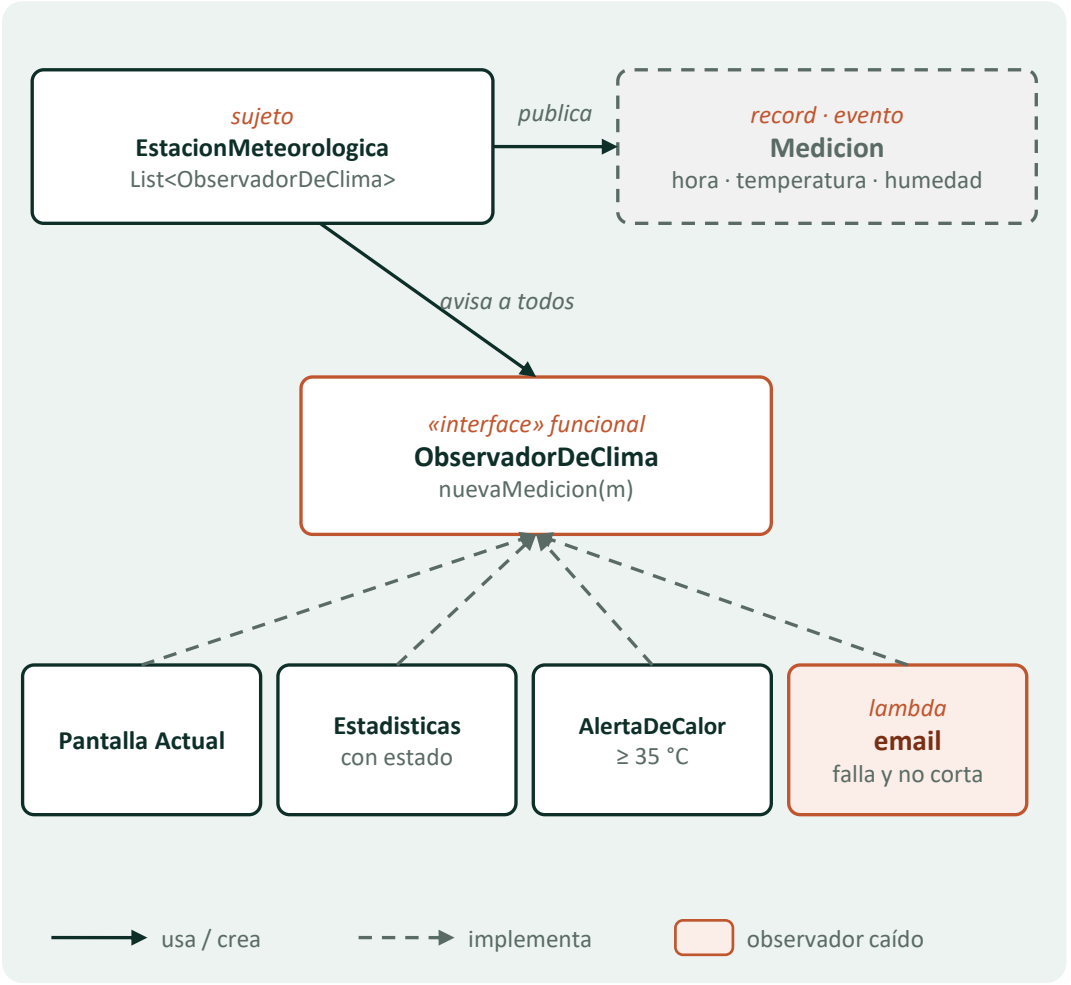
Intención *Que un objeto avise a muchos interesados cuando cambia, sin conocerlos.*

El problema

Cada medición de la estación la usan la pantalla, las estadísticas, una alerta de calor y un resumen por email. Mañana se suma un registro histórico. La estación no debería cambiar por eso.

ANTES · sin el patrón

```
public void registrar(Medicion m) {
    pantalla.mostrar(m);
    estadisticas.acumular(m);
    if (m.temperatura() >= 35) alertas.avisar(m);
    email.enviarResumen(m);
    // cada interesado nuevo = modificar la estación
    // y si el email está caído, se corta todo
}
```



7

Observer en código

DESPUÉS

```
@FunctionalInterface
public interface ObservadorDeClima {
    void nuevaMedicion(Medicion medicion);
}

public final class EstacionMeteorologica {
    private final List<ObservadorDeClima> observadores;

    public EstacionMeteorologica() {
         this.observadores = new CopyOnWriteArrayList<>();
    }
    public void suscribir(ObservadorDeClima o) {
         observadores.add(o);
    }
    public void desuscribir(ObservadorDeClima o) {
         observadores.remove(o);
    }
     public void publicar(Medicion medicion) {
        for (ObservadorDeClima o : observadores) {
            try { o.nuevaMedicion(medicion); }
            catch (RuntimeException e) { registroDeErrores.accept(...); }  // no corta
        }
    }
}

estacion.suscribir(new PantallaActual());
estacion.suscribir(new Estadisticas());          // tiene estado propio: clase
estacion.suscribir(new AlertaDeCalor(35));
estacion.suscribir(m -> enviarResumen(m));       // simple: lambda
```

```
$ ./run.sh observer

estación publica 13:00:
  pantalla -> 36,5 °C · 35 %
  estadísticas -> máx 36,5 · prom 30,3
  ALERTA -> 36,5 °C supera 35 °C
  (!) email caído -> se sigue
```

Cuándo NO usarlo

- Un único interesado fijo: una llamada directa se lee y se depura mejor.
- En Spring no lo hagas a mano: ApplicationEventPublisher y @EventListener.

8

Facade

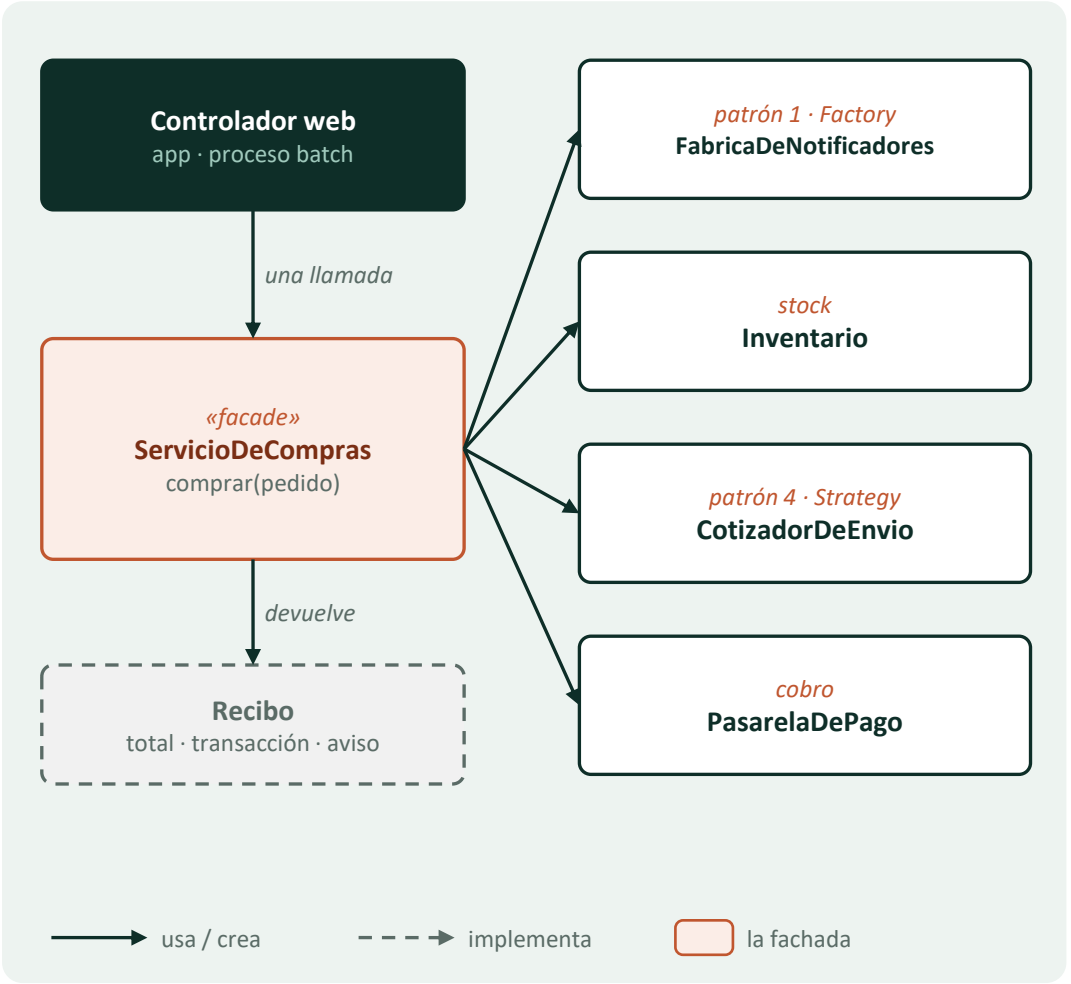
Intención *Ofrecer una interfaz simple a un conjunto de subsistemas.*

El problema

Comprar exige reservar stock, cotizar el envío, cobrar, devolver el stock si el pago falla y avisar al cliente, en ese orden. El controlador web, la app y un proceso batch no deberían repetirlo.

ANTES · sin el patrón

```
@PostMapping("/compras")
public Recibo comprar(@RequestBody PedidoDto dto) {
    // reservar stock... cotizar envío... cobrar...
    // si el pago falla, devolver el stock...
    // avisar al cliente... armar el recibo...
    // 70 líneas en el controlador, repetidas en el
    // endpoint de la app y en el proceso batch
}
```



8

Facade en código

DESPUÉS

```
public final class ServicioDeCompras {
    private final Inventario inventario;
    private final CotizadorDeEnvio cotizador;              // Strategy (patrón 4)
    private final PasarelaDePago pasarela;
    private final FabricaDeNotificadores notificadores;    // Factory Method (patrón 1)

    public Recibo comprar(Pedido p) {
        Notificador aviso = notificadores.crear(p.canal());   // 0. falla antes de tocar nada
        inventario.reservar(p.sku(), p.cantidad());                          // 1. stock
        BigDecimal productos = inventario.precio(p.sku()).multiply(valueOf(p.cantidad()));
        BigDecimal envio = cotizador.cotizar(p.opcionEnvio(), p.paquete());  // 2. envío
        BigDecimal total = productos.add(envio);

        PasarelaDePago.Resultado pago = pasarela.cobrar(total, p.tokenTarjeta()); // 3. cobro
        if (!pago.aprobado()) {
            inventario.liberar(p.sku(), p.cantidad());          // compensación
            throw new CompraRechazadaException("Pago rechazado: se liberó el stock");
        }
        String enviado = aviso.enviar(p.cliente(), "Compra confirmada");       // 4. aviso
        return new Recibo(p.cliente(), p.sku(), p.cantidad(), productos, envio, total,
                pago.idTransaccion(), enviado);
    }
}
```

```
$ ./run.sh facade

1 Ana, 1 teclado    [OK] $ 48.700
2 Bruno, 2 (hay 1)  [X] sin stock
3 Carla sin fondos  [X] stock liberado
4 Diego, retiro     [OK] $ 45.000
5 Eva, canal FAX    [X] antes de cobrar
stock final: TECLADO=0
```

Cuándo NO usarlo

- Si solo reenvía a un único objeto, es una capa de más.
- Si crece hasta saber todo del sistema, es un God object: una fachada por caso de uso.

PARA SU PROYECTO

Del ejemplo en Java puro a Spring Boot

Patrón	En el ejemplo	En el proyecto con Spring Boot
Factory	`fabrica.registrar("WHATSAPP", …)`	Cada Notificador es un @Component; se inyecta Map<String, Notificador> (la clave es el nombre del bean)
State	`estado = estado.aprobar()`	Igual: es dominio puro. Se persiste el nombre del estado en una columna y se reconstruye al leer (@Convert)
Chain	`Aprobador.cadena(…)`	Los eslabones son beans; un @Configuration arma la cadena o se inyecta List<Aprobador> con @Order
Strategy	`cotizador.agregar("Express", …)`	Cada estrategia es un @Component con nombre; el cotizador pide Map<String, EstrategiaDeEnvio>
Repository	`…EnMemoria + replace atómico`	Spring Data JPA con @Version; el conflicto llega como ObjectOptimisticLockingFailureException
Adapter	`new AdaptadorEstacionXml(…)`	Un @Component por proveedor, elegido con @Profile o @ConditionalOnProperty
Observer	`estacion.suscribir(observador)`	ApplicationEventPublisher + @EventListener (o @TransactionalEventListener si hay base de datos)
Facade	`DemoFacade arma todo a mano`	@Service con @Transactional en comprar(): si el pago falla, el rollback devuelve el stock

Cuatro patrones más y dos advertencias

Builder

CREACIONAL

Objetos con muchos campos opcionales: un pedido, un DTO de reporte, un filtro de búsqueda. En el proyecto: @Builder de Lombok o un record.

Decorator

ESTRUCTURAL

Caché, log o reintentos alrededor de un repositorio o de un adaptador, sin tocarlos. Misma interfaz; se apilan en el orden que importa.

Template Method

COMPORTAMIENTO

Exportar un reporte a CSV, PDF o Excel: mismo esqueleto (consultar, armar, escribir) y un paso distinto por formato.

Command

COMPORTAMIENTO

Acciones con deshacer, o tareas guardadas en una cola para ejecutarlas o reintentarlas más tarde (por ejemplo, sin conexión).

Singleton

NO LO ESCRIBAN A MANO

Lo resuelve el contenedor: todo bean de Spring es singleton por defecto. El getInstance() estático complica los tests.

Abstract Factory

SOLO SI HAY FAMILIAS

Sirve para familias de objetos que deben ir juntas (proveedores por país, temas de interfaz). Sin familias, Factory Method alcanza.

Cómo saber si el patrón está bien aplicado

1. **El patrón resuelve un problema que pueden nombrar**
   Si no pueden decir qué cambio futuro absorbe, sobra.

2. **Está escrito el costo**
   Más clases y más indirección: anótenlo en el ADR junto con la alternativa descartada.

3. **Hay un test por regla**
   Y si hay concurrencia, un test con varios hilos y un resultado exacto.

4. **Se llegó refactorizando, en pasos**
   Test que fija el comportamiento → refactor → patrón, en commits separados.

5. **Saben cuándo NO usarlo**
   Poder decir dónde no lo aplicaron, y por qué, también es diseño.

Preguntas para defender cada patrón

- ¿Qué cambio futuro absorbe este patrón?
- ¿Qué archivos hay que tocar para agregar un caso nuevo?
- ¿Qué pasa si dos usuarios lo hacen a la vez?
- ¿Cómo lo testean sin la base ni la API real?
- ¿Qué alternativa descartaron, y por qué?

PARA LLEVARSE

Tres ideas y una consigna

1

Un patrón resuelve un problema

Si no pueden nombrar el problema que resuelve, sobra.

2

Todo patrón tiene un costo

Más clases y más indirección. Decirlo en voz alta es parte del diseño.

3

La concurrencia la garantiza el almacenamiento

@Version, UNIQUE o un lock. El patrón ordena el código; el motor garantiza.

Para la próxima clase: corran ./run.sh, rompan un test a propósito y elijan los tres patrones que van a defender en su ADR.

¿Preguntas?